ADVANCED WAZUH SOC/XDR PLATFORM

Detailed Implementation & Engineering Plan

Product build sequence, interfaces, testing and research execution

1. Engineering Doctrine

Build the product as an external modular layer around Wazuh. Keep **wazuh-main** as source/reference and **wazuh-docker** as deployment infrastructure. The custom product lives in **advanced_wazuh_soc**. Do not modify Wazuh core unless a concrete requirement cannot be satisfied through supported integration mechanisms.

2. Current Baseline

- Wazuh 4.14.7 is running in Docker.
- Manager, Indexer and Dashboard are operational.
- Python virtual environment is configured.
- FastAPI service is operational.
- Wazuh REST API authentication has been verified.
- The project has a modular repository structure.

3. Repository Architecture

```
advanced_wazuh_soc/
backend/ — API, settings, orchestration
normalizer/ — schemas and event transformation
correlation/ — entity/time/context correlation
ml/ — feature engineering, training, inference, evaluation
threat_intel/ — provider abstraction, cache, provenance
risk_engine/ — deterministic/contextual risk
attack_chain/ — timeline and ATT&CK; reasoning
response/ — recommendations, approvals, safe actions
dashboard/ — project-specific analyst UI
configs/ — validated configuration
datasets/ — labeled research data
experiments/ — benchmark runners and results
tests/ — unit/integration/security tests
docs/ — specifications, architecture and evidence
```

4. Phase 0 — Repository and Configuration

- Create configuration/settings module.
- Validate environment variables at startup.
- Create `.env.example` without secrets.
- Add logging configuration.
- Add lint/test tooling.
- Add README explaining Wazuh/custom boundary.

Acceptance: a new developer can start the custom service without changing Wazuh source and without exposing credentials.

5. Phase 1 — Wazuh Integration

- Implement Wazuh API client.
- Authenticate and obtain JWT programmatically.
- Add timeout/retry policy.
- Implement alert retrieval with pagination/limits.
- Normalize API errors into application errors.
- Write mocked unit tests.
- Add live integration test for `/` and manager metadata.
- Add alert retrieval endpoint.

Acceptance: application retrieves Wazuh metadata and alerts reliably, and secrets/tokens never appear in logs.

6. Phase 2 — Ingestion and Normalization

- Define schema version 1.
- Map Wazuh fields to normalized fields.
- Preserve raw alert payload.
- Normalize timestamps to UTC internally.
- Extract entities.
- Validate schema with Pydantic or equivalent.
- Build fixtures for representative Wazuh alerts.
- Measure field coverage.

Acceptance: every supported fixture produces deterministic normalized output with source provenance.

7. Phase 3 — Correlation Engine

- Implement configurable sliding windows.
- Create entity indexes for IP, host, user and hash.
- Implement duplicate detection.
- Implement rule-based relationship predicates.
- Create incident lifecycle: open, update, close.
- Preserve alert-to-incident relationships.
- Add confidence to relationships.
- Test multi-step attack scenarios and benign repeated events.

Acceptance: related alerts become a coherent incident while unrelated events remain separate.

8. Phase 4 — Threat Intelligence

- Implement provider interface rather than coupling core logic to one vendor.
- Select one provider for MVP.
- Cache indicator results.
- Apply timeout/rate-limit controls.
- Record source, confidence, lookup timestamp and indicator type.
- Treat CTI as enrichment, not unquestioned truth.

- Test positive, negative and unavailable-provider cases.

Acceptance: CTI improves context without becoming a single point of failure.

9. Phase 5 — Risk Engine

Start with a deterministic score so the system remains interpretable before ML. Candidate factors include Wazuh severity, incident alert count, confirmed CTI match, entity reputation/context, attack stage and asset criticality when available. Document weights and thresholds. Every score must expose its factors.

Acceptance: identical input produces identical deterministic risk output.

10. Phase 6 — ML Prioritization

1. Collect authorized benign and attack telemetry.
2. Define labels and ground-truth rules.
3. Split data temporally or by scenario to prevent leakage.
4. Create a simple baseline.
5. Train candidate tree-based models if data supports them.
6. Calibrate thresholds on validation data.
7. Evaluate only on held-out test data.
8. Generate explanations.
9. Persist model metadata and feature schema.
10. Integrate inference behind a stable interface.

Acceptance: model performance is reproducible and compared against a non-ML baseline.

11. Phase 7 — Attack Chain

- Map supported alerts to ATT&CK; techniques.
- Order contributing evidence by time.
- Represent entities and relationships.
- Identify transitions between tactics/techniques only when evidence supports them.
- Generate timeline plus confidence.
- Never invent missing steps.
- Expose source alerts for every reconstructed step.

Acceptance: a reviewer can trace every attack-chain node back to one or more alerts.

12. Phase 8 — Response Recommendation

- Define response taxonomy.
- Map incident conditions to candidate actions.
- Generate rationale and evidence.
- Require approval for high-impact actions.
- Log approvals and outcomes.
- Integrate Active Response only after safety tests.
- Implement dry-run mode before execution.

Acceptance: no high-impact action can execute unexpectedly.

13. Phase 9 — Investigation Interface

- Incident queue sorted by risk.
- Incident details and contributing alerts.
- Timeline/attack-chain visualization.
- Entity and IOC context.
- Risk-factor explanation.
- Recommendation/approval panel.
- Audit history.
- Filtering/search by host, user, IP, rule and time.

14. Phase 10 — Production Hardening

- Containerize custom services.
- Add health/readiness checks.
- Add structured logs and metrics.
- Pin dependencies.
- Add dependency/security scanning.
- Add backup/recovery procedures.
- Document configuration and secret rotation.
- Load-test the pipeline.
- Perform API and authorization tests.

15. Research Experiment Matrix

Experiment	System	Purpose	Primary measurements
E0	Wazuh only	Baseline	Detection, FP, latency
E1	Wazuh + correlation	Incident grouping value	Alert reduction, correlation precision
E2	Wazuh + ML	Prioritization value	Precision/recall/F1/FPR, ranking
E3	Full system	Combined value	All operational metrics
E4	Ablation	Component contribution	Delta vs full system

16. Research Dataset Protocol

- Use authorized lab telemetry only.
- Record attack scenario, start/end time and expected behavior.
- Record benign workload conditions.
- Preserve raw source evidence.
- Assign labels using documented ground truth.
- Separate training, validation and test data.
- Version datasets and preprocessing code.
- Record environment configuration and software versions.

17. Performance Testing

Measure ingestion rate, normalization latency, correlation latency, ML inference latency, end-to-end latency, CPU, RAM and storage growth. Start with the laptop/local environment. Do not claim enterprise throughput until a representative benchmark is executed.

18. Failure Testing

- Invalid Wazuh credentials.
- Expired JWT.
- Wazuh API unavailable.
- Malformed alert.
- CTI provider unavailable.
- ML model unavailable.
- Database unavailable.
- Duplicate event replay.
- Clock/timestamp irregularity.
- Unauthorized response request.

19. Release Gates

1. Requirements reviewed.
2. Unit/integration/security tests passing.
3. Secrets excluded from repository.
4. API contracts documented.
5. Wazuh/custom boundary verified.
6. Performance benchmark recorded.
7. Research experiment reproducible.
8. Known limitations documented.
9. Release tagged with configuration/model/schema versions.

20. Productization Roadmap

Local MVP: reliable Wazuh integration and incident intelligence.
V1: ML, CTI, attack chain, investigation interface and controlled recommendations.
V2: additional data sources, UEBA, enterprise identity, observability, multi-host deployment and selective automation.
Future: managed deployment, broader integrations, stronger case management and optional AI-assisted hunting.

21. What We Deliberately Defer

- Kubernetes before there is a measured scaling requirement.
- Kafka before simpler ingestion is proven insufficient.
- LLM-based hunting before deterministic search/investigation is reliable.
- Broad cloud integrations before the core incident model is stable.
- Automatic destructive response before dry-run and approval workflows are proven.
- Claims of production-scale performance without representative benchmarks.

22. Final Engineering Principle

Every layer should answer three questions: **What evidence did we receive? What transformation or inference did we perform? Why should an analyst trust the resulting conclusion?** This principle connects product quality, security engineering and research validity.